

serif

Practical No.12

Aim: To implement Euclid's Algorithm for finding GCD (Greatest Common Divisor) using both iterative and recursive approaches, including the Extended Euclidean Algorithm.

Theory: Euclid's Algorithm is an efficient method for computing the greatest common divisor (GCD) of two numbers. The algorithm is based on the principle that the GCD of two numbers also divides their difference.

Theorem: $\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$ **Base Case:** $\text{GCD}(a, 0) = a$

The Extended Euclidean Algorithm not only finds the GCD of two numbers a and b , but also finds integers x and y such that: $ax + by = \text{GCD}(a, b)$

This is useful in various applications including:

- Finding modular multiplicative inverse
- Solving linear Diophantine equations
- RSA encryption/decryption

Code:

```
// Euclid's Algorithm for Finding GCD - TypeScript Implementation
// Theorem:  $\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$ 
// Base case:  $\text{GCD}(a, 0) = a$ 

// Find GCD using Euclid's algorithm (iterative approach)
function gcdIterative(a: number, b: number): number {
    // Ensure non-negative inputs
    a = Math.abs(a);
    b = Math.abs(b);

    while (b !== 0) {
        console.log(` GCD(${a}, ${b}) = GCD(${b}, ${a} mod ${b} = ${a % b})`);
        [a, b] = [b, a % b];
    }

    console.log(` GCD(${a}, 0) = ${a}`);
    return a;
}

// Find GCD using Euclid's algorithm (recursive approach)
function gcdRecursive(a: number, b: number, depth: number = 0): number {
    // Ensure non-negative inputs
    a = Math.abs(a);
    b = Math.abs(b);

    const indent = ' '.repeat(depth);

    if (b === 0) {
```

```
    if (b === 0) {
      return [a, 1, 0];
    }

    // Recursive call
    const [g, x1, y1] = extendedGCD(b, a % b);

    // Update x and y using results of recursive call
    const x = y1;
    const y = x1 - Math.floor(a / b) * y1;

    return [g, x, y];
  }

  // Extended Euclidean Algorithm (iterative approach)
  function extendedGCDIterative(a: number, b: number): [number, number, number] {
    // Ensure non-negative inputs
    a = Math.abs(a);
    b = Math.abs(b);

    // Initialize
    let oldR = a, r = b;
    let oldS = 1, s = 0;
    let oldT = 0, t = 1;

    while (r !== 0) {
      const quotient = Math.floor(oldR / r);
      [oldR, r] = [r, oldR - quotient * r];
      [oldS, s] = [s, oldS - quotient * s];
      [oldT, t] = [t, oldT - quotient * t];
    }

    // oldR = GCD, oldS and oldT are coefficients
    return [oldR, oldS, oldT];
  }

  // Helper function: GCD without printing steps
  function gcdIterativeQuiet(a: number, b: number): number {
    a = Math.abs(a);
    b = Math.abs(b);

    while (b !== 0) {
      [a, b] = [b, a % b];
    }

    return a;
  }
```

```
// Find GCD of multiple numbers using Euclid's algorithm
function gcdMultiple(...numbers: number[]): number {
  if (numbers.length === 0) {
    throw new Error('At least one number is required');
  }

  let result = Math.abs(numbers[0]);
  for (let i = 1; i < numbers.length; i++) {
    result = gcdIterativeQuiet(result, Math.abs(numbers[i]));
  }

  return result;
}

// Find Least Common Multiple (LCM) using GCD
// Formula: LCM(a, b) = |a × b| / GCD(a, b)
function lcm(a: number, b: number): number {
  if (a === 0 || b === 0) return 0;
  return Math.abs(a * b) / gcdIterativeQuiet(a, b);
}

// Verify the given example: GCD(16, 12) = 4
function verifyExample(): number {
  console.log('=' .repeat(70));
  console.log('VERIFICATION OF GIVEN EXAMPLE');
  console.log('=' .repeat(70));
  console.log('\nExample: Find GCD(16, 12)');
  console.log('-' .repeat(70));

  const a = 16, b = 12;

  console.log('\nMethod 1: Iterative Approach');
  console.log('  Steps:');
  const result = gcdIterative(a, b);

  console.log('\n  Result: GCD(${a}, ${b}) = ${result}');

  console.log('\n' + '-' .repeat(70));
  console.log('\nMethod 2: Recursive Approach');
  console.log('  Steps:');
  const resultRec = gcdRecursive(a, b);

  console.log('\n  Result: GCD(${a}, ${b}) = ${resultRec}');

  console.log('\n' + '-' .repeat(70));
  console.log('\n VERIFIED: GCD(${a}, ${b}) = ${result}');

  return result;
}
```

```
}

// Demonstrate various GCD calculations
function demonstration(): void {
  console.log('\n' + '='.repeat(70));
  console.log('ADDITIONAL EXAMPLES');
  console.log('='.repeat(70));

  const examples: [number, number][] = [
    [12, 4],
    [48, 18],
    [1071, 462],
    [0, 5],
    [17, 0],
    [0, 0]
  ];

  for (const [a, b] of examples) {
    console.log('\nExample: GCD(${a}, ${b})');
    console.log('-'.repeat(40));
    const result = gcdIterativeQuiet(a, b);
    console.log('Result: GCD(${a}, ${b}) = ${result}');
  }

  // Demonstrate GCD of multiple numbers
  console.log('\n' + '='.repeat(70));
  console.log('GCD OF MULTIPLE NUMBERS');
  console.log('='.repeat(70));

  console.log('\nExample: GCD(48, 36, 24, 12)');
  const result = gcdMultiple(48, 36, 24, 12);
  console.log('Result: ${result}');

  // Demonstrate LCM
  console.log('\n' + '='.repeat(70));
  console.log('LEAST COMMON MULTIPLE (LCM)');
  console.log('='.repeat(70));

  console.log('\nExample: LCM(12, 18)');
  const resultLCM = lcm(12, 18);
  console.log('Formula: LCM(12, 18) = |12 × 18| / GCD(12, 18)');
  console.log('LCM(12, 18) = 216 / ${gcdIterativeQuiet(12, 18)}');
  console.log('Result: LCM(12, 18) = ${resultLCM}');
}

// Demonstrate Extended Euclidean Algorithm
function extendedGCDDemo(): void {
  console.log('\n' + '='.repeat(70));
```

```
console.log('EXTENDED EUCLIDEAN ALGORITHM');
console.log('='.repeat(70));

console.log('
The Extended Euclidean Algorithm finds:
1. GCD(a, b)
2. Coefficients x, y such that:  $ax + by = \text{GCD}(a, b)$ 

Applications:
- Finding modular multiplicative inverse
- Solving linear Diophantine equations
- RSA encryption/decryption
');

const examples: [number, number][] = [
  [16, 12],
  [240, 46],
  [56, 15]
];

for (const [a, b] of examples) {
  console.log(`\nExample: Extended GCD(${a}, ${b})`);
  console.log('='.repeat(50));

  const [g, x, y] = extendedGCD(a, b);
  console.log('Results:');
  console.log(`  GCD(${a}, ${b}) = ${g}`);
  console.log(`  Coefficients: x = ${x}, y = ${y}`);
  console.log(`  Verification:  $\${a}(\${x}) + \${b}(\${y}) = \${a * x + b * y}$ `);
  console.log(`  Expected GCD: ${g} `);
}

// Main function
function main(): void {
  // Verify example
  verifyExample();

  // Show more examples
  demonstration();

  // Show extended GCD
  extendedGCDDemo();
}

// Run if executed directly
main();
```

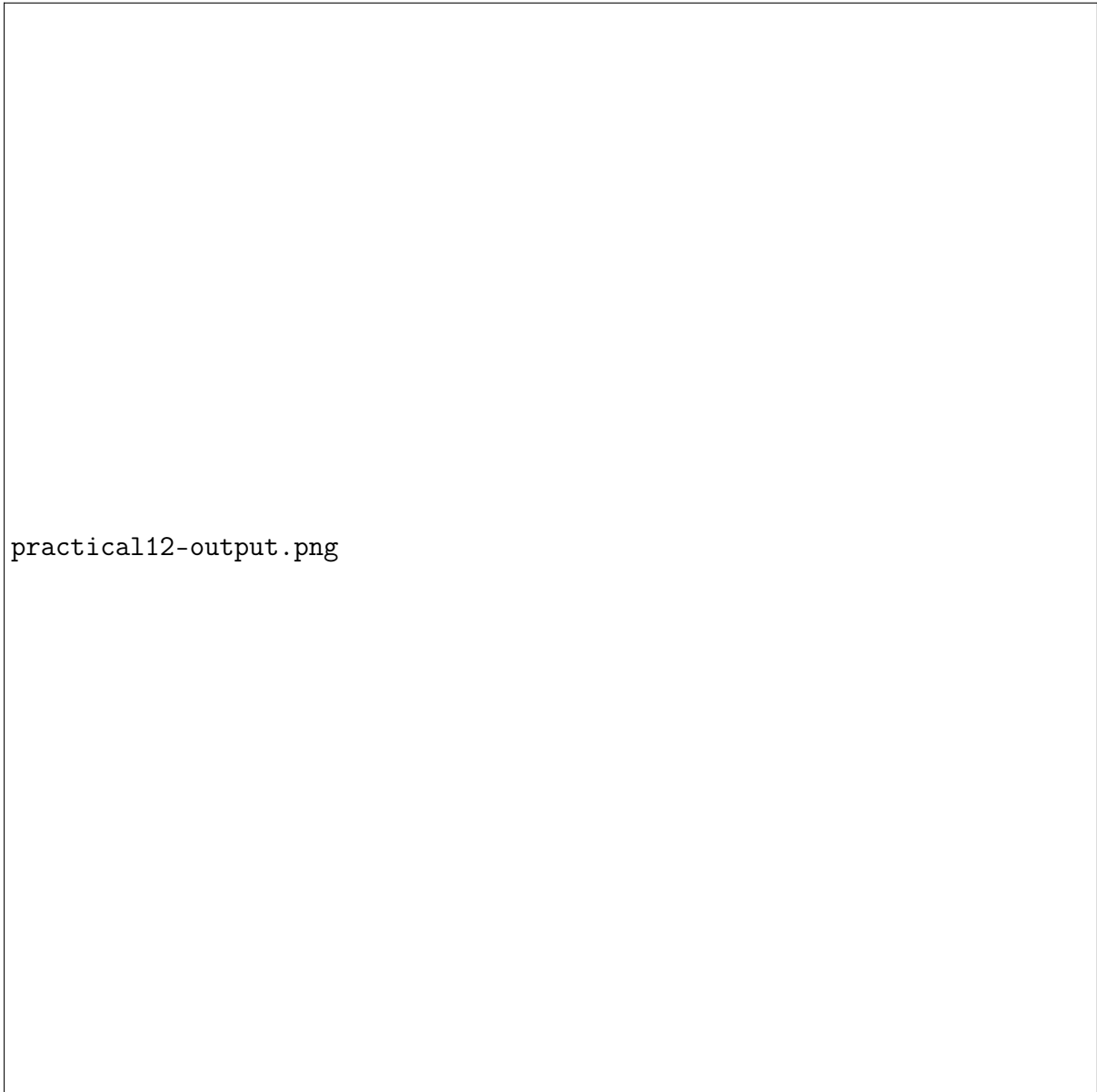
Output:

Figure 1: Output of Euclid's Algorithm Implementation